

swagger.bot – Project Overview

Purpose and Core Functionality

swagger.bot is a TypeScript-based tool designed to automatically discover and retrieve publicly available **OpenAPI (Swagger) specification files** for a given application or API. In an era where APIs are proliferating across domains, this tool addresses the need to quickly locate the machine-readable API descriptions that many services expose. OpenAPI/Swagger specs serve as blueprints of how to interact with an API – detailing endpoints, operations, and data models. According to a 2024 study of SaaS platforms, OpenAPI (formerly Swagger) is the most widely supported format for API specifications. This means countless services already publish or utilize these specs, but finding them isn't always straightforward.

The core purpose of **swagger.bot** is to **streamline API discovery**: given the name of an app, service, or developer API, the bot will search across the public web to find any corresponding OpenAPI (or Swagger 2.0) spec file (e.g. often available as JSON or YAML files like `swagger.json` or `openapi.yaml`). By locating these specs, **swagger.bot** helps developers and integrators skip the manual hunting process and jump straight into understanding or using the API. This is especially useful for enabling **unified access to APIs** across different domains – if every API can be represented in a standard OpenAPI format, tools can consume them in a uniform way. (In fact, *“many synchronous web APIs already use [OpenAPI] to describe themselves”*, so **swagger.bot** leverages that common practice.)

Importantly, **swagger.bot** doesn't rely on a static database of API specs alone (though it can complement those). Whereas projects like APIs.guru's OpenAPI Directory compile known API specs from public sources, **swagger.bot** actively **searches the live web** for specification files on-demand. For example, if asked to find the API spec for “ExampleCRM,” the bot might try multiple strategies – querying search engines for `<ExampleCRM> API openapi.json`, scanning the company's developer documentation for mentions of Swagger, or checking common URLs (like `api.examplecrm.com/docs` or `.well-known/openapi.json`). The goal is to cast a wide net across the public internet to discover any openly published API definitions. Once found, the tool can return the raw spec or even provide summarized insights from it (more on that below).

Ultimately, by quickly finding OpenAPI definitions for disparate services, **swagger.bot supports the larger vision of unified API access across domains**. If each service's capabilities are captured in a standardized spec, then integrating with multiple services becomes much easier – one can programmatically read the spec and know how to call that API. This aligns with the mission of platforms like Unified.to, which focus on *unifying* multiple APIs behind a consistent

interface. In other words, swagger.bot is a discovery mechanism that finds the “keys” (the API specs) that unlock access to many services in a uniform way.

System Architecture and LangGraph Orchestration

The architecture of swagger.bot is built around a modular, multi-step **workflow** orchestrated by **LangGraph JS**. LangGraph is a low-level framework for building controllable AI agents and workflows. It allows us to define a directed process (a *graph*) where each node can perform a specific task – in this case, tasks related to finding, processing, and responding with API specs. Using LangGraph, swagger.bot’s logic is organized into distinct phases with clear inputs/outputs, making the overall system more reliable and maintainable. The **LangGraph pipeline** manages the flow from the initial query (the app or API name) through to the final result (the located spec or its summary), handling decision-making and error-checking along the way.

At a high level, the swagger.bot workflow can be broken down into three main stages: **Discovery**, **Enrichment**, and **Response**. Below is an overview of each stage and how the system operates:

1. **Discovery Phase – Finding the API Specification:** The process begins when a user (or calling program) provides the name of an application or API they’re interested in. LangGraph coordinates a set of **tools** or sub-routines to search for the corresponding OpenAPI/Swagger file. This may involve:
 - **Web Search Tool:** The agent uses a search utility (possibly via a search API or custom web scraper) to find likely URLs or documentation pages. It might search for keywords like “<ProductName> API swagger.json” or “<Service> OpenAPI specification”. If the target is a well-known SaaS or platform, it often leads to a developer portal or GitHub repository containing the spec.
 - **Common Path Probing:** In some cases, the agent may try known standard endpoints. For example, some services host their spec at a standard URL (`https://api.service.com/v1/swagger.json` or under a `.well-known` path). The bot can attempt such URLs programmatically.
 - **APIs Directory Lookup:** If relevant, it can cross-check known open API directories (for instance, the APIs.guru dataset or other collections) to see if the target API is listed there. This provides a shortcut for popular APIs.

During discovery, LangGraph ensures the agent can iterate and adapt. If the first attempt yields no result, the graph might branch or loop: e.g. try a second search query with alternate terms (maybe the company renamed their API product, etc.), or scan the main website for any JSON links. Thanks to LangGraph’s orchestrated design, these retries and logic branches are systematically handled rather than ad-hoc. The output of the discovery

phase is one or more candidate spec URLs or files. In an ideal case, the bot finds a direct link to an OpenAPI definition (JSON/YAML). If multiple candidates are found (say, both a `swagger.yaml` and a Postman collection link), the system can rank them (potentially using AI reasoning or simple heuristics like file format) to choose the best match.

2. **Enrichment Phase – *Parsing and Enhancing the Spec***: Once a spec file is located, swagger.bot proceeds to process it. The raw OpenAPI or Swagger file is machine-readable (JSON or YAML structure). In this stage, the tool:

- **Fetches the Spec Content**: Using an HTTP client, it retrieves the spec file from the discovered URL. Any format (OpenAPI v3 or Swagger v2, JSON or YAML) is accepted – the system can parse either.
- **Parses and Validates**: The spec content is parsed into a structured data model in memory. This might use existing libraries or Node SDKs for OpenAPI parsing to ensure the file is valid. If parsing fails (e.g., the URL was not actually a spec or the content is malformed), the agent can fallback to discovery to try another candidate.
- **Enhances the Data**: Here's where enrichment can involve AI assistance. For example, the tool can generate a human-friendly summary of the API's capabilities: what the API primarily does, how many endpoints or major resources it has, and noteworthy requirements (auth, etc.). It could also cross-reference if the spec is incomplete – e.g., if it doesn't include examples, the system might mark that or even attempt to fill in descriptions using AI. This step transforms the raw spec into something more immediately useful for the end-user.

Integration with AI tools is often utilized in this phase: For instance, swagger.bot can send the spec (or parts of it) to an LLM (Large Language Model) to get a concise overview in natural language. This is especially helpful if the end goal is not just to retrieve the file, but to understand it. Modern LLMs (like OpenAI's GPT-4 or Anthropic's Claude) are capable of reading the structured spec and summarizing its contents or answering specific questions about it. We highlight these AI integrations in the next section. The enriched output could be a summary report (e.g., "**ExampleCRM API** – 12 endpoints covering customer and lead management, requires OAuth2 authentication, returns JSON; notable endpoints include ..."), or even a transformed version of the spec (for example, filtered to just certain sections relevant to the query).

3. **Response Phase – *Delivering the Result***: In the final stage, LangGraph assembles the results to return to the user. Depending on usage context, the response could be:

- The raw OpenAPI spec file itself (or a direct link to it), if the goal is to feed it into another tool or to allow the user to download it.
- A **summary or report** derived from the spec, if the user wants a human-readable overview. For example, an integrator might ask "Does **ExampleCRM** have an API and

what does it offer?” – swagger.bot would then provide a synopsis if it found the spec (or indicate if no spec was found).

- An interactive **analysis** if integrated into a chat or QA system – e.g., swagger.bot could answer follow-up questions about the API by referencing the spec it found. (This essentially transforms the static API documentation into an interactive knowledge source, similar to an “OpenAPI chatbot.”)

The LangGraph framework ensures that all relevant data from previous stages is available when constructing the response. It can incorporate the original query, the discovered spec content, and the AI-generated insights into a coherent answer. Because the logic is defined in a graph, the system can also handle cases like multiple APIs found (maybe the query was broad, like a category of APIs) by iterating or aggregating results. The end result is then returned to the user in the requested format (which could be JSON output, a markdown answer, or any format suitable for a README or documentation page).

Throughout these stages, using **LangGraph JS** offers significant architectural benefits. LangGraph allows **customizable agent behavior and memory**, meaning swagger.bot can maintain context between steps (e.g., remembering which URLs were already tried) and can be extended with complex logic without sacrificing clarity. The framework’s emphasis on reliability (even enabling human-in-the-loop if needed for verification) ensures that even complex discovery tasks can be handled in a controlled manner. In short, LangGraph acts as the conductor for the swagger.bot orchestra – each component (search, parse, AI, etc.) is a player that LangGraph queues up and directs, resulting in a smooth flow from question to answer.

AI Integration and Intelligent Processing (OpenAI, LangChain, Claude)

A key aspect of swagger.bot is its integration with **AI tools and language models** to augment the discovery and understanding of API specs. While the basic function (finding the spec file) doesn’t strictly require AI, using AI greatly enhances what the system can do with the spec once found, and even how it finds it. Here are the main integration points with AI and how they are used in swagger.bot:

- **Natural Language Query Understanding:** If a user’s query is unstructured (for example, “Find me the Swagger file for the main Salesforce API”), an AI model can be used to interpret nuances. In most cases the query is straightforward, but an LLM could help normalize the input (e.g., recognizing that “Salesforce” likely refers to the Salesforce REST API and its spec might be at a known URL). This is a minor use, as often a direct search is sufficient, but it’s an example of understanding intent.
- **Spec Summarization:** Once an OpenAPI spec is retrieved, swagger.bot can call an **LLM (Large Language Model)** to summarize it in plain English. Models like **OpenAI’s GPT-4** or

Anthropic's Claude are adept at processing large text (including JSON/YAML) and extracting the salient points. For instance, the system might prompt the model with something like: *"Here is the OpenAPI specification for ExampleCRM's API. Summarize the key endpoints and functionalities, and note any special requirements."* The model's response would then be included in the output, giving users a quick understanding without reading the entire spec. This is particularly useful if the spec is very lengthy or detailed – the AI can highlight important sections (auth methods, rate limits, major resource groups, etc.). By leveraging such models, swagger.bot essentially provides an **AI assistant on top of API documentation**, akin to having a smart documentation parser. (This concept is similar to what others have done with "OpenAPI bots" that answer questions about an API from its spec, but here it's seamlessly part of the discovery tool.)

- **Reasoning and Validation:** AI can also help in reasoning over multiple findings. Suppose the discovery phase returns two possible spec files for a given name (maybe one is a legacy version and one is current). The system can employ an LLM to **compare and decide** which spec is more relevant. The model could be asked to read brief excerpts or titles of each spec and infer which one the user likely wants. Additionally, if a spec is partially incomplete or ambiguous, an AI could attempt to infer missing pieces. For example, if the spec lacks descriptions for endpoints, an LLM might generate descriptions based on endpoint names (flagging them as AI-generated). While this doesn't replace official docs, it enriches the spec to be more useful.
- **LangChain Toolkit Integration:** Although swagger.bot is built with LangGraph, it can still integrate with **LangChain** components where convenient. LangChain provides an `OpenAPIToolkit` and agent utilities that let an LLM directly use an OpenAPI spec to decide which API calls to make. In swagger.bot's context, we might use LangChain's tooling to do things like *test an endpoint* or *answer a specific question* about the API by effectively simulating calls. For example, if the user asks, "How do I create a new contact using the ExampleCRM API?", the system could use LangChain's OpenAPI agent with the found spec to identify the correct endpoint and required parameters, and then either return that info or even perform a dummy call (if appropriate and safe). This kind of integration is optional but highlights how the found specs enable powerful AI-driven interactions. LangChain or similar frameworks can serve as the "decision-making brain" on top of the raw data that swagger.bot discovers.
- **Model Choices (GPT-4, Claude, etc.):** The project is model-agnostic to an extent – it can work with any AI service that offers a suitable API. By default, OpenAI's GPT series might be used (due to their strong capabilities with text), but Anthropic's Claude is equally suitable, especially known for handling larger context windows which can be helpful for big API specs. In practice, we can allow pluggable AI backends. For instance, a developer using swagger.bot can configure their OpenAI API key or Claude key, and the system will use that for all LLM queries. There might even be scenarios to use one model for certain tasks and another model for others (e.g., a simpler model for scanning text vs. a more

powerful one for complex summarization). The integration points are designed such that swapping the model or provider requires minimal changes – thanks to abstraction either through LangGraph’s tool interface or using a bridging library like LangChain which supports multiple LLM providers.

In summary, AI integration transforms swagger.bot from a simple finder of API specs into an **intelligent API assistant**. It ensures that not only will users get the raw technical files they need, but they can also get **immediate insights and answers** from those files. This can significantly speed up the evaluation of an API. Instead of manually reading a 50-page JSON spec to find out if an API supports a certain feature, a user can ask swagger.bot in plain language and get an answer, with the confidence that it’s sourced from the official specification. (It’s worth noting that because swagger.bot ultimately relies on official specs as the source of truth, the answers it gives about the API are grounded in the actual documentation, which is a form of retrieval-augmented generation rather than the model making up info.)

Optional Gradio Interface for Demo & Debugging

While the core of swagger.bot is a backend service/agent, the project also provides an **optional UI layer** using **Gradio**. Gradio is a lightweight web interface library that allows developers to turn Python or JavaScript functions into interactive web apps with minimal effort. In the context of swagger.bot, a Gradio interface can serve as a simple front-end to demonstrate and debug the tool’s capabilities.

For example, the repository might include a Gradio app where a user can input the name of an API (via a text box) and then trigger the swagger.bot pipeline. The interface would then display the results, such as:

- The name of the spec file found (or an error if not found).
- Perhaps the raw spec content or a download link.
- The AI-generated summary or analysis of the spec.
- Logs or intermediate steps (useful for debugging, might be shown in an expanded accordion or console panel).

Using Gradio for showcasing means that anyone can spin up a local web demo of swagger.bot without writing additional UI code – ideal for testing queries or sharing the tool with non-developers. It also helps during development: the maintainers can quickly try out different APIs in a browser and visually inspect how the agent is performing, which tools it’s invoking, etc. Gradio’s components (like chat boxes or code display blocks) make it easy to present complex outputs (for instance, showing a formatted snippet of the spec, or a collapsible section for each step the agent took).

It's important to note that the Gradio interface is **optional** and primarily for convenience. In production or continuous use, swagger.bot would likely run as a service (perhaps triggered by an API call or integrated into another system), rather than through a Gradio UI. However, including the Gradio demo aligns with good development practice: providing a quick way to manually verify and demonstrate the project's functionality. This can also double as a debugging tool – if a user reports that a certain API's spec isn't being found, the developers can use the Gradio UI to replicate the query and observe where the process might be failing (for example, maybe a particular site requires a special header to fetch the spec, etc.).

In summary, Gradio adds a **human-friendly face** to swagger.bot, useful for demonstrations (e.g., showing the project in a tech talk or to a team) and for interactive debugging of the AI components. It underscores the project's commitment to not just operate behind the scenes, but also to be accessible for developers to experiment with directly.

Vision: Enabling Unified API Access Across Domains

One of the driving motivations behind swagger.bot is to facilitate **unified API access**. The idea of a unified API is that a single interface can allow integration with multiple services, abstracting away the differences between each provider's individual API. By collecting standardized specs from across the web, swagger.bot becomes an important piece in achieving that vision.

Figure: An illustration of the unified API concept. A single unified interface (center “Unified API” gear) connects to multiple underlying service APIs (right), allowing an application (left) to integrate with many services through one mechanism.

Having OpenAPI specifications for many different services means one can generate connectors or adapters for each service in a uniform manner. For instance, if you have specs for Salesforce, HubSpot, and Zoho CRM, you could programmatically generate client code or define a common interface for all three (since each spec tells you what endpoints and data models exist). Tools like OpenAPI Generator or Postman can take an OpenAPI spec and produce SDKs automatically. In our context, swagger.bot supplies the raw material (the specs) needed to build such unification layers.

This vision aligns closely with what **Unified.to** and similar “unified API” platforms aim to do. Unified.to itself is an “AI-first unified API platform” that brings together over *200+ integrations across 16+ categories behind a single API*. In other words, Unified.to provides a central API that internally connects to many different third-party APIs (for HR systems, CRM systems, etc.), so developers only have to code against Unified.to's API rather than dozens of individual ones. For a platform like that, having accurate and up-to-date API specifications for each integrated service is crucial – it ensures the unified interface can map correctly to each underlying API's endpoints and data.

swagger.bot can be seen as a complementary tool in this ecosystem. Its **goal is to gather the standardized specs** that describe each service, which can then feed into a unified API layer or any integration framework. By discovering publicly available Swagger/OpenAPI files, it reduces the manual work needed to onboard a new integration. Instead of reading through documentation pages and writing a spec from scratch, a developer (or an automated system) can use swagger.bot to fetch the official spec and then adapt it for the unified interface. Even if some adjustments are needed (perhaps the unified API wants to standardize field names or authentication flows across services), having the baseline spec is a huge head start.

The **benefits of unified API access** are well-known: faster development, reduced complexity, and a consistent developer experience across different services. By contributing to the pool of standardized API descriptions, swagger.bot is indirectly pushing towards those benefits. Imagine a future where for any SaaS product or service, you could query a bot like swagger.bot and immediately get an OpenAPI spec. That spec could then be fed into a tool that automatically plugs it into a unified schema – resulting in near-instant integration. While we're not fully there yet, this project is a step in that direction.

In practical terms, swagger.bot's integration with the **Unified.to SDK** hints at how it can work within a unified API platform's context. The Unified.to SDK allows programmatic access to their platform (managing connections, routing calls, etc.). Once swagger.bot finds an external API's spec, Unified.to's tools could be used to set up a "connection" to that API through their unified endpoints. For example, if swagger.bot discovers the OpenAPI spec for a new CRM service, Unified.to could use that spec to quickly map the CRM's endpoints to Unified.to's common data model, potentially even generating portions of the integration code automatically. Unified.to's *Passthrough API* is one feature that could benefit – it allows passing calls through to an underlying API via a unified interface. With a known spec, configuring such passthroughs becomes simpler because you know exactly what endpoints exist and what payloads they expect.

In essence, swagger.bot empowers the unified API vision by **bridging the discovery gap**. It finds the raw specifications that describe each API in detail. Those specs, in turn, enable tools and platforms (like unified API providers) to do the actual unification – whether that's normalizing data models or consolidating authentication. By focusing on the discovery and standardization piece, swagger.bot contributes to an ecosystem where any API can be accessible in a plug-and-play fashion. This emphasis on standardized specs and cross-domain integration is what makes the project more than just a utility – it's part of a broader effort to **make diverse APIs more accessible and interoperable**.

Repository Structure and Key Components

The swagger.bot project's codebase is organized in a way that reflects its functional components, making it easy to navigate for contributors and reviewers. Below is a summary of

the repository structure and its core parts:

- `/src` – Main source code directory (TypeScript). Most of the implementation lives here, divided into sub-modules:
 - `src/agents/` **or** `src/graph/` – *LangGraph workflow definitions*. This is where the LangGraph JS configuration is defined, orchestrating the discovery→enrichment→response sequence. It may include a JSON or TypeScript-based description of the agent's graph. For example, a file like `swaggerBot.graph.ts` could instantiate the LangGraph nodes and their connections. This layer ties everything together.
 - `src/discovery/` – *Spec discovery utilities*. Contains functions or classes responsible for searching the web and locating spec URLs. For instance, a `WebSearchTool` module might wrap a search engine API call; a `SpecLocator` service might implement known URL pattern checks. If APIs.guru or other datasets are consulted, that logic resides here too. This sub-module outputs candidate links for specs.
 - `src/fetcher/` – *HTTP client and parser*. Code that takes a URL (from discovery) and retrieves the content. It likely uses `fetch` or `Axios` to GET the file. It then parses JSON or YAML. Parsing might leverage a library (e.g., `js-yaml` for YAML, or OpenAPI-specific parsers). The output is a normalized JavaScript object representation of the API spec (often following the OpenAPI JSON structure). Validation logic also lives here – e.g., ensuring required fields like `openapi` or `swagger` version identifier exist in the JSON.
 - `src/enrichment/` – *Enrichment and AI processing*. This includes any code that interacts with AI models or adds value to the raw spec. For example, a `SpecSummarizer.ts` that takes the parsed spec and uses an LLM (via an SDK call) to generate a summary. It might also include helper functions to extract certain info from the spec (like a list of endpoints or tags) which can be formatted nicely for output. If multiple AI models or prompts are used (one for summary, one for Q&A), their prompt templates and calls would be defined here.
 - `src/outputs/` – *Response formatting*. This could contain logic to format the final result. For example, converting the summary and any spec metadata into a Markdown string (for README output) or into a structured JSON (if being used as an API). If `swagger.bot` is intended to be used via a command-line, this might also include CLI output formatting. In a LangGraph context, this might be part of the final node's responsibility, but organizing formatting helpers in one place keeps things tidy.
 - `src/unified/` – *Unified.to SDK integration*. Any code specifically dealing with the Unified.to platform or SDK would be here. For instance, if after finding a spec, the tool optionally tries to create a Unified.to integration object, that logic would be encapsulated here. Or if the Unified.to SDK is used to fetch something (though primarily, we expect it to be used for making calls via the unified API), those calls are

defined in this module. This separation means the core logic (finding specs) doesn't become entangled with Unified.to specifics – it's an add-on.

- `src/ui/` or `/demo ****` – *Gradio interface and other UIs*. If the project includes the Gradio demo, the code for launching that app (e.g., a `app.ts` or `demo.tsx` file that sets up Gradio interfaces) would be here. This might import functions from the main modules to run when a user clicks the “Search” button on the web UI. In addition to Gradio, if there are other ways to interface (maybe a simple Express server for an API endpoint, or a CLI script entrypoint), those could be placed here or in a `/bin` directory.
- `/test` – Test cases for the project (if implemented). For example, unit tests might exist for the discovery module (ensuring that given a known site, it finds the correct URL) or for the parser (checking that a known swagger.json is parsed correctly). Integration tests could simulate an end-to-end run for a few well-known APIs.
- `/docs` – Documentation assets. This might include additional markdown files, images, or sample spec files. If this overview were part of the repository README, it would reside at the root, but supplementary docs (like design decisions or an architecture diagram) could be here.
- **Configuration Files** – At the repository root, there would be typical config files:
 - `package.json` with project metadata and dependencies (likely including `@langchain/langgraph`, `unified.to` SDK, OpenAPI parser libs, and possibly OpenAI/Anthropic SDKs or a generic LLM SDK).
 - `langgraph.config.json` or similar, if LangGraph uses a config file to define agent parameters or if deploying to a LangGraph cloud service.
 - `.env.example` indicating environment variables (for instance, API keys for OpenAI or others, which would not be committed but documented).
 - `README.md` – which would contain a user-friendly description (a shorter version of this overview, installation and usage instructions, etc.).

Each of these components works in concert as described in the architecture section. For example, when running the tool, the LangGraph-defined agent in `src/graph` will invoke functions from `src/discovery` to search, use `src/fetcher` to get the file, possibly call `src/enrichment` to process it, and then produce an output via `src/outputs`. This separation of concerns makes the codebase easier to extend. If someone wanted to add support for another spec format (say, Postman collections or GraphQL SDL), they could add a new discovery routine and parser without altering the core flow. Or if a new AI model comes out, the integration in `src/enrichment` can be swapped or improved independently.

The repository is structured to facilitate **project planning and collaboration**. Each module's responsibility is clear, which aligns with how one would plan tasks: e.g., one developer can work on improving the web search logic while another works on better summarization prompts, without stepping on each other's toes. The README (or project overview) ties everything

together, ensuring that new contributors or stakeholders can understand the high-level idea (as this document demonstrates) and then drill down into the part of the codebase relevant to their interest.

swagger.bot is an evolving project at the intersection of API discovery and AI-driven understanding. By combining a robust orchestration framework (LangGraph) with the power of standardized API descriptions and modern AI models, it aims to simplify the way developers find and make sense of APIs across the web. Whether used as a standalone utility or as part of a larger unified API ecosystem, swagger.bot's ultimate goal is to reduce friction in API integration – enabling a world where disparate services can be connected through the common language of OpenAPI specs.